



How to Build a Satellite Image Classifier and Detect Deforestation ...

How to Build a Satellite Image Classifier and Detect Deforestation Using Machine Learning

For more free resources, follow me on IG @collegewith_hari

This guide walks you through training a machine learning model to classify satellite images by land type, then using that model to detect where deforestation is happening by comparing images from different years. Everything you need is free. If you've never written a line of code before, this guide still works. I explain every step from scratch.

What Machine Learning Actually Is (Quick Version)

Machine learning is when you show a computer thousands of examples of something and it learns the pattern on its own. Instead of writing rules like "if the image is mostly green, it's a forest," you show the model 3,000 images of forests and 3,000 images of farmland and it figures out the difference by itself.

The specific type of machine learning you're using here is called image classification. You're training a model to look at a satellite image and answer the question: what type of land is this?

The model you're using is called a convolutional neural network (CNN). Without getting too deep into the math, a CNN works by scanning an image in small sections (called filters), looking for patterns like edges, textures, shapes, and color combinations. The first layers detect simple things like straight lines and color gradients. The deeper layers combine those simple patterns into more complex ones, like "this combination of green texture plus irregular edges probably means forest." By the time the image reaches the final layer, the model outputs a prediction: forest, farmland, river, city, etc.

You don't need to understand every layer of the network to use it. You just need to know how to set it up, feed it data, and evaluate the results. That's what this guide covers.

What Transfer Learning Is and Why You're Using It

Training a CNN from scratch requires millions of images and days of computing time. You don't have either of those. What you do have is access to models that have already been trained on millions of images by researchers at places like Microsoft and Google.

Transfer learning means you take one of these pretrained models and adapt it for your specific task. The model you're using is called ResNet50. It was trained on a dataset called ImageNet, which contains over 14 million images across 1,000 categories (dogs, cars, buildings, food, etc.). Through that training, ResNet50 learned how to recognize general visual features: edges, textures, shapes, color patterns, spatial relationships.

Those general features are useful for satellite images too. A texture pattern is a texture pattern whether it's in a photo of a dog or a satellite image of a forest. So instead of starting from zero, you start with a model that already knows how to see, and you just teach it the last step: what these specific 10 types of land look like from space.

Practically, this means you freeze all the layers of ResNet50 except the very last one. You replace that last layer with a new one that outputs 10 categories instead of 1,000. Then you train only that last layer on your satellite images. The rest of the model stays exactly as it was. This is why you can get 95%+ accuracy in under an hour of training instead of spending days building a model from scratch.

The Dataset: EuroSAT



How to Build a Satellite Image Classifier and Detect Deforestation ...

It contains 27,000 satellite images taken by the European Space Agency's Sentinel-2 satellite. Sentinel-2 orbits Earth and captures imagery of the surface every 5 days. Each image in the dataset is a 64x64 pixel patch of land at 10 meters per pixel resolution, meaning each pixel represents a 10m x 10m area on the ground.

Every image is labeled as one of 10 land cover categories:

1. Annual Crop: fields that get replanted each season, like wheat, corn, or rice
2. Forest: dense tree cover
3. Herbaceous Vegetation: natural grasslands, meadows, non-crop ground cover
4. Highway: major roads and highways
5. Industrial: factories, warehouses, power plants, commercial zones
6. Pasture: land used for grazing livestock
7. Permanent Crop: orchards, vineyards, things that stay planted for multiple years
8. Residential: neighborhoods, housing developments, suburbs
9. River: rivers and streams
10. Sea/Lake: large bodies of water

Each class has between 2,000 and 3,000 images. The dataset comes in two versions: an RGB version (just the red, green, and blue bands, like a normal photo) and a full 13-band multispectral version. For this project, use the RGB version. It's simpler to work with and still gets you excellent accuracy.

Download from any of these (they're all the same data):

Kaggle: kaggle.com/datasets/apollo2506/eurosat-dataset

GitHub: github.com/phelber/EuroSAT

Hugging Face: huggingface.co/datasets/blanchon/EuroSAT_RGB

The dataset is about 90MB. Download the RGB JPEG version.

Setting Up Your Environment

You need three things: Python, PyTorch (a machine learning library), and a GPU to train on. You don't need to install any of them on your computer. Google Colab gives you all three for free in your browser.

Go to colab.research.google.com. Sign in with a Google account. Click "New Notebook." You now have a Python environment running on Google's servers.

First thing you do: switch to a GPU. In the top menu, click Runtime, then Change Runtime Type, then select T4 GPU from the dropdown. Click Save. Without this step, training will take hours instead of minutes. The GPU is what does the heavy lifting during training.

Now upload your dataset. The easiest way is to upload the EuroSAT folder to your Google Drive, then connect your Drive to Colab:

```
from google.colab import drive
drive.mount('/content/drive')
```

It will ask you to sign in to your Google account and give permission. After that, your entire Google Drive is accessible from the Colab notebook at `/content/drive/MyDrive/`.

If you don't want to use Drive, you can download the dataset directly in Colab using the Kaggle API. But Drive is simpler for beginners.

Now install any extra libraries you need. PyTorch and torchvision come pre-installed in Colab. If you want to use fastai (a library that simplifies a lot of the PyTorch boilerplate code), run this in a cell:

← How to Build a Satellite Image Classifier and Detect Deforestation ...

The EuroSAT dataset comes organized into folders by class. It looks like this:

```
EuroSAT_RGB/
  AnnualCrop/
    AnnualCrop_00001.jpg
    AnnualCrop_00002.jpg
    ...
  Forest/
    Forest_00001.jpg
    Forest_00002.jpg
    ...
  Highway/
    Highway_00001.jpg
    ...
  (and so on for all 10 classes)
```

This folder structure matters because PyTorch has a built-in function called `ImageFolder` that automatically reads images from this structure and assigns labels based on the folder names. You don't need to manually label anything. The folder name IS the label.

Splitting the Data

You never train and test on the same data. If you do, the model just memorizes the images instead of learning the patterns, and it fails on anything it hasn't seen before. This is called overfitting.

Split the 27,000 images into three groups:

Training set (80%, about 21,600 images): the model learns from these

Validation set (10%, about 2,700 images): checked after each training cycle to see if the model is actually learning or just memorizing

Test set (10%, about 2,700 images): held out until the very end for your final accuracy number

In PyTorch:

```
from torchvision import datasets
from torch.utils.data import random_split
```

```
full_dataset = datasets.ImageFolder(root='path/to/EuroSAT_RGB', transform=your_transforms)
```

```
train_size = int(0.8 * len(full_dataset))
```

```
val_size = int(0.1 * len(full_dataset))
```

```
test_size = len(full_dataset) - train_size - val_size
```

```
train_set, val_set, test_set = random_split(full_dataset, [train_size, val_size, test_size])
```

The `random_split` function randomly assigns each image to one of the three groups. This randomness is important because you don't want all the easy images in training and all the hard ones in testing.

Preprocessing the Images

Before feeding images to ResNet50, you need to do two things: resize them and normalize them.

Resizing: ResNet50 expects input images of 224x224 pixels. Your EuroSAT images are 64x64. The resize operation scales them up. This might seem counterintuitive (you're making a small image bigger, not adding



How to Build a Satellite Image Classifier and Detect Deforestation ...

Normalization: this is where most beginners get confused but it's actually simple. Every pixel in an image has a value between 0 and 255 for each color channel (red, green, blue). Normalization rescales those values so they center around zero with a standard deviation of one. You use the exact mean and standard deviation values from ImageNet because that's what ResNet50 was trained on:

```
mean = [0.485, 0.456, 0.406]
std = [0.229, 0.224, 0.225]
```

These numbers correspond to the red, green, and blue channels respectively. If you don't normalize with these exact values, the pretrained weights won't work properly because the model expects inputs in this specific range.

In PyTorch, you define transforms like this:

```
from torchvision import transforms
```

```
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

```
test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

Notice that the training transform includes `RandomHorizontalFlip` and `RandomVerticalFlip`. These are data augmentation techniques. Each time a training image is loaded, there's a 50% chance it gets flipped horizontally and a 50% chance it gets flipped vertically. Satellite images can be viewed from any orientation (there's no "upside down" when you're looking at Earth from space), so flipping doesn't change the content but it effectively gives the model more variation to learn from. You don't apply augmentation to test or validation images because you want to evaluate on the original, unmodified images.

Apply `train_transform` to your training set and `test_transform` to your validation and test sets.

Creating Data Loaders

Data loaders are how PyTorch feeds images to the model in batches. Instead of loading all 21,600 training images into memory at once (which would crash your session), it loads them in groups of 32 or 64 at a time.

```
from torch.utils.data import DataLoader
```

```
train_loader = DataLoader(train_set, batch_size=32, shuffle=True)
val_loader = DataLoader(val_set, batch_size=32, shuffle=False)
test_loader = DataLoader(test_set, batch_size=32, shuffle=False)
```

`shuffle=True` for training means the order of images is randomized each time. This prevents the model from learning patterns based on the order images appear rather than the actual content.

Building the Model

Load the pretrained ResNet50 and modify the final layer:



How to Build a Satellite Image Classifier and Detect Deforestation ...

```
model = models.resnet50(pretrained=True)
```

```
for param in model.parameters():
    param.requires_grad = False
```

```
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, 10)
```

```
model = model.to('cuda')
```

Here's what each line does:

`models.resnet50(pretrained=True)` downloads ResNet50 with all its pretrained ImageNet weights. This is a one-time download, about 100MB.

The for loop freezes every layer in the model. `requires_grad = False` means "don't update these weights during training." This keeps all the learned visual features intact.

`model.fc` is the final fully connected layer. Originally it outputs 1,000 classes (for ImageNet). You replace it with a new layer that outputs 10 classes (your 10 land types). This new layer is NOT frozen, so it WILL be updated during training.

`model.to('cuda')` moves the model to the GPU. If you skip this, everything runs on CPU and training takes much longer.

Training the Model

Set up the loss function, optimizer, and training loop:

```
import torch
import torch.optim as optim
```

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.fc.parameters(), lr=0.001)
```

```
num_epochs = 10
```

```
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
```

```
    for images, labels in train_loader:
        images = images.to('cuda')
        labels = labels.to('cuda')
```

```
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

```
    running_loss += loss.item()
```

```
model.eval()
correct = 0
total = 0
```



How to Build a Satellite Image Classifier and Detect Deforestation ...

```

labels = labels.to('cuda')
outputs = model(images)
_, predicted = torch.max(outputs, 1)
total += labels.size(0)
correct += (predicted == labels).sum().item()

val_accuracy = correct / total
print(f'Epoch {epoch+1}/{num_epochs}, Loss: {running_loss/len(train_loader):.4f}, Val Accuracy: {val_accuracy:.4f}')

```

Here's what's happening:

CrossEntropyLoss is the standard loss function for classification problems. It measures how wrong the model's predictions are. The goal of training is to make this number as small as possible.

Adam optimizer is what actually updates the model's weights to reduce the loss. The learning rate (lr=0.001) controls how big each update step is. Too high and the model overshoots. Too low and it learns too slowly. 0.001 is a standard starting point.

model.fc.parameters() means you're only optimizing the final layer. Everything else is frozen.

Each epoch is one full pass through all 21,600 training images. After each epoch, the model runs through the validation set to check accuracy. You should see the loss going down and the accuracy going up each epoch.

Expected results after 10 epochs: validation accuracy around 91-95%.

Fine-Tuning for Higher Accuracy

After training the final layer, you can unfreeze the entire model and train all layers at a much lower learning rate. This lets the earlier layers slightly adjust their features to better match satellite imagery instead of the natural photos they learned from.

```

for param in model.parameters():
    param.requires_grad = True

```

```
optimizer = optim.Adam(model.parameters(), lr=0.0001)
```

Run another 5-10 epochs with this setup. The learning rate is 10x lower (0.0001 vs 0.001) because you don't want to destroy the pretrained features, just gently adjust them.

Expected results after fine-tuning: validation accuracy around 95-98%.

If validation accuracy stops improving while training loss keeps dropping, you're overfitting. Stop training at that point. The best model is usually the one from the epoch right before accuracy started to plateau or decline.

Evaluating Your Model

After training, run the model on the test set and report these metrics:

```

from sklearn.metrics import classification_report, confusion_matrix
import numpy as np

```

```

model.eval()
all_preds = []
all_labels = []

```


← How to Build a Satellite Image Classifier and Detect Deforestation ...

```
outputs = model(images)
_, predicted = torch.max(outputs, 1)
all_preds.extend(predicted.cpu().numpy())
all_labels.extend(labels.numpy())
```

```
class_names = ['AnnualCrop', 'Forest', 'HerbVeg', 'Highway', 'Industrial', 'Pasture', 'PermCrop', 'Residential',
'River', 'SeaLake']
```

```
print(classification_report(all_labels, all_preds, target_names=class_names))
```

This gives you precision, recall, and F1-score for every class plus overall accuracy. Some classes will perform better than others. Forest and Sea/Lake usually get near-perfect accuracy because they have very distinct visual patterns. Highway and River sometimes get confused because both appear as long thin features cutting through the landscape. Pasture, Herbaceous Vegetation, and Annual Crop can overlap because they're all green ground cover from above.

Document these patterns. Explaining where your model struggles and why is what separates a tutorial from a research project. Admissions officers and journal reviewers specifically look for this type of critical analysis.

Generate a confusion matrix to visualize the errors:

```
import matplotlib.pyplot as plt
import seaborn as sns

cm = confusion_matrix(all_labels, all_preds)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', xticklabels=class_names, yticklabels=class_names)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.savefig('confusion_matrix.png')
plt.show()
```

Save this figure. It goes in your paper.

Turning This Into a Research Paper (Detecting Deforestation)

Training the classifier is the foundation. Applying it to detect real-world change is what makes it research.

Step 1: Pick a study area. Choose a region where deforestation is documented. Southeast Asia, the Amazon, Central Africa, or even areas in your own country. You need a region where you can find satellite imagery from two different time periods.

Step 2: Download satellite imagery. Go to the Copernicus Open Access Hub (scihub.copernicus.eu) or Google Earth Engine (earthengine.google.com). Both are free. Create an account. Search for Sentinel-2 imagery of your study area from two dates, for example 2018 and 2024. Download the RGB bands.

Step 3: Chop the satellite images into patches. Your model expects 64x64 pixel inputs. Write a script that divides the full satellite image into a grid of 64x64 patches. In Python with PIL:

```
from PIL import Image
img = Image.open('sentinel_image.tif')
width, height = img.size
patches = []
for i in range(0, width - 64, 64):
    for j in range(0, height - 64, 64):
        patch = img.crop((i, j, i + 64, j + 64))
```



How to Build a Satellite Image Classifier and Detect Deforestation ...

(x, y) position of each patch. Now you have a land cover map, a grid showing what type of land exists at each location.

Do this for both time periods (2018 and 2024). You now have two land cover maps of the same area.

Step 5: Compare the maps. For each position, check if the classification changed between the two dates. Flag every patch that was "Forest" in 2018 but became "Annual Crop," "Pasture," "Industrial," or "Residential" in 2024. Those are deforestation events detected by your model.

Step 6: Validate your results. Go to Global Forest Watch (globalforestwatch.org), which publishes annual tree cover loss data based on independent analysis. Download their deforestation data for your study area and compare it to what your model found. Calculate the overlap: what percentage of deforestation events that your model detected were also reported by Global Forest Watch? What did your model catch that they missed? What did they report that your model missed?

That comparison is the core finding of your paper. You trained a lightweight model, applied it to real imagery, and benchmarked it against the industry standard. Agreement validates your approach. Disagreement is interesting because it tells you where your model needs improvement or where official data might have gaps.

Writing the Paper

Abstract: 150-250 words. State the problem (deforestation monitoring requires efficient automated tools), your method (transfer learning with ResNet50 on EuroSAT, applied to a specific region), your key results (classification accuracy, number of deforestation events detected, agreement rate with Global Forest Watch), and your conclusion.

Introduction: why deforestation monitoring matters (cite a few statistics on global forest loss rates), what existing methods exist (manual satellite interpretation, Global Forest Watch's methodology), and what your contribution is (applying a lightweight, reproducible CNN approach to a specific region and validating against official data).

Methods: describe the EuroSAT dataset, the ResNet50 architecture, the transfer learning procedure, the training hyperparameters, the study area you chose, the time periods, how you preprocessed the Sentinel-2 imagery, and how you performed the change detection. Be specific enough that a reader could exactly replicate your experiment.

Results: test set accuracy (overall and per-class), confusion matrix, the land cover change maps, the number and location of detected deforestation events, and the comparison with Global Forest Watch data. Use figures: the confusion matrix, a map showing detected changes, and a side-by-side comparison with official data.

Discussion: what worked, what didn't, and why. Common limitations to acknowledge: your model was trained on European imagery (EuroSAT) but applied to a tropical or different region, which could cause accuracy differences. 64x64 patches can miss small-scale deforestation. RGB-only imagery loses spectral information that could help distinguish vegetation types. Cloud cover in satellite imagery can cause misclassifications. Seasonal changes in vegetation (crops harvested, deciduous trees losing leaves) could be falsely flagged as land cover change. Acknowledging these honestly shows scientific maturity.

Conclusion: summarize your findings and suggest improvements. Using all 13 spectral bands instead of RGB. Training on imagery from the same geographic region as the study area. Adding temporal sequences instead of just two snapshots. These are real next steps that future work could address.

Where to Submit

Journal of Emerging Investigators (emerginginvestigators.org): free to publish, reviewed by Harvard scientists. Good fit if your project has an environmental science angle.

← How to Build a Satellite Image Classifier and Detect Deforestation ...

Columbia Junior Science Journal (columbiajuniorsciencejournal.org). about 5% acceptance rate. very competitive but carrying the Columbia name on a publication is significant.

Science fairs: enter at the regional level. Strong projects advance to state and potentially ISEF. Regeneron STS deadline is November 5 if your project is strong enough.

All Tools and Links

EuroSAT dataset (Kaggle): kaggle.com/datasets/apollo2506/eurosat-dataset

EuroSAT dataset (GitHub): github.com/phelber/EuroSAT

EuroSAT dataset (Hugging Face): huggingface.co/datasets/blanchon/EuroSAT_RGB

Google Colab (free GPU): colab.research.google.com

PyTorch documentation: pytorch.org/docs

fastai library: docs.fast.ai

Copernicus Open Access Hub (Sentinel-2 imagery): scihub.copernicus.eu

Google Earth Engine (satellite imagery): earthengine.google.com

Global Forest Watch (validation data): globalforestwatch.org

scikit-learn (evaluation metrics): scikit-learn.org

Matplotlib/Seaborn (visualization): matplotlib.org, seaborn.pydata.org

Original EuroSAT paper: search "EuroSAT Helber 2019 IEEE" on Google Scholar